Studio of Ten

onboarding runbook

// shape b shared-deploy for partners #2 through #10

Phase 2 Tenant Deployment Checklist

VERSION
v1.0 · 2026-07

PUBLISHED
2026-07-03

STUDIO
studiooften.com

Phase 2 Tenant Deployment Checklist

Status: LIVE as of 2026-07-02. First real tenant onboarding = Melisa (pilot zero). **Companion to:** MELISA_PILOT_ZERO_DEPLOYMENT_RUNBOOK.md (Shape A · separate deploy).

Use this checklist when onboarding a new tenant to the shared multi-tenant platform (Shape B · single-deploy, multiple tenants). Shape A separate-deploy pattern from the Melisa runbook is still valid for pilot zero; Shape B is what unlocks partners #2 and beyond.

Phase 2 pipeline shipped 2026-07-02 across commits `cc7d05c6 · 239655d0 · 213c109d · 31303127 · 86372a33 · 07119254 · f31a990e · 851091e3`. See `src/lib/tenant-resolver.ts` inline documentation for architecture.

2026-07-03 surgical brand de-hardcode (commit `c8bc00bc`). Four tenant-illusion-breaking JSX/HTML sites now route through `brand()` instead of hardcoded "Body Recode": - `src/app/dashboard/layout.tsx` — coach dashboard header (name + auto-derived badge initials) - `src/app/payment-success/page.tsx` — client-facing after Stripe - `src/app/client/[token]/page.tsx` — client portal - `src/app/api/coach/approve-checkin-response/route.ts` — HTML status page returned when a coach clicks Approve

2026-07-03 full de-hardcode pass (commit `b51abc89`). Codemod-driven — 278 mutations across 100 files. Routes exact `Body Recode` JSX text + string literals + logo `alt` attributes through `brand().name`, and remaining email addresses / URLs through `coach()` / `brand()` / `appUrl()`. Kade-only dashboards + help guide skipped (they reference BR as one of Kade's 5 brands, not the current tenant). Deferred (post-pilot): compound brand strings like "Body Recode Performance Coaching" and "Body Recode Playbook" — these need per-string decisions on whether they're product names (keep) vs tenant-swappable text.

2026-07-03 custom domain support (commit `22f3acf9`). New `tenant_domains` table (schema at `~/Dropbox/01_BODY_RECODE/06_SAAS_PLATFORM_BUILD/sql/2026-07-03_tenant_domains_schema.sql`, applied). Resolver extended with `NEXT_PUBLIC_TENANT_DOMAIN_MAP` env var fast-path for edge middleware. `/api/tenant/domains` GET/POST/DELETE + new domains section in `/dashboard/settings/tenant`. Add-a-domain flow returns the env var line ready for copy-paste into Vercel.

2026-07-03 Stripe Connect foundation (commit `17da8ffd`). Phase 3 kickoff. `TenantConfig.licence` gains optional `stripeAccountId` + `stripeAccountStatus`. Helpers at `src/lib/tenant-stripe.ts`. Onboarding endpoints: POST `/api/tenant/stripe/onboard` (creates Standard account + AccountLink) + GET `/api/tenant/stripe/callback` (polls status). StripeConnectSection in tenant settings UI. **BR path untouched** — all 15 existing checkout callsites keep using `process.env.STRIPE_SECRET_KEY` directly. Deferred: per-callsite refactor to route charges

to tenant Connect accounts (needs per-flow decisions on platform vs tenant billing) + `/api/webhooks/stripe/connect` for Connect account events.

---

01 Pre-flight

- [ ] Tenant signed the licence agreement + paid setup deposit (per PARTNER_JOURNEY Stage 3)
- [ ] Business name locked (used as `brand.name` + auto-derived `tenantId`)
- [ ] Modality confirmed (strength or yoga — the built modality packs)
- [ ] Brand assets in hand: logo (light + dark), accent color hex, tagline
- [ ] Domain: owned + DNS access confirmed
- [ ] Stripe account created for their business (separate from BR Stripe)
- [ ] Resend account configured with their sending domain (DKIM/SPF verified)

02 Step 1 · Provision the tenant

Option A · Self-serve via /signup form (fastest)

1. Direct tenant to `https://bodyrecode.au/signup`
2. Fill in: full name, business name, email, password, modality
3. Optional: custom tenant slug (auto-derived if blank)
4. Submit → API creates auth user + tenant_config row with default values
5. Confirmation screen shows their `tenantId` + `coachId`

Option B · Manual SQL provisioning (for special cases)

Use the seed template at `~/Dropbox/01_BODY_RECODE/06_SAAS_PLATFORM_BUILD/sql/TENANT_SEED_TEMPLATE.sql` — replace the placeholder values with the tenant's data + run via `supabase db query --linked`.

---

03 Step 2 · Tenant configures their brand

Tenant logs in at `/portal/login` (their own coach email + password) then navigates to `/dashboard/settings/tenant`. Every section is editable inline:

- **Brand shell** (13 fields — name, logos, domains, email addresses, accent color)
- **Coach identity** (10 fields — name, photo, credentials, social handles, WhatsApp)
- **Product wrapping** (11 fields — product names + prices in AUD)
- **Licence** (`tenantId` , `poweredBy` toggle, `version` stamp)
- **Modality** (id + label + doctrineMode — usually leave at A for founding partners)

Saves via `POST /api/tenant/update` which: - Verifies caller is authenticated + on coach allowlist - Uses RLS-protected client (coach can only update their own row) - Invalidates the in-memory tenant cache on success

04 Step 3 · Domain routing

Two approaches depending on tenant's setup.

Approach 1 · Subdomain on shared deployment (fastest, cheapest)

1. Point their subdomain (e.g. `melisa.sot-platform.com`) at the BR Vercel deployment via CNAME
2. `src/lib/tenant-resolver.ts` already recognises 3-part hosts — extracts first label as tenant_id (`melisa.sot-platform.com` → `tenant_id='melisa'`)
3. No middleware changes needed
4. Test: `curl -H "Host: melisa.sot-platform.com" https://bodyrecode.au/` should render Melisa's brand once feature flag is on

Approach 2 · Custom domain (future — tenant_domains table)

Not yet implemented. Requires a `tenant_domains` mapping table + resolver extension. Track for post-Founding-Ten evergreen tier.

05 Step 4 · Enable DB-backed tenant config

In Vercel project settings → Environment Variables:

- `NEXT_PUBLIC_TENANT_DB_ENABLED=true`

Redeploy. On next request: - Middleware sets `x-tenant-id` header based on host - Root layout calls `prefetchTenant(tenantId)` → warms cache - Every downstream `getTenant()` returns THEIR config, not BR's

Falls back to hardcoded BR config on any DB failure. Zero-risk cutover.

06 Step 5 · Wire tenant Stripe

- [ ] Tenant creates their Stripe products (Report, Challenge, Blueprint, Membership) matching what's in their `products` config
- [ ] Tenant enables Stripe Connect if they'll receive payments directly (not-yet-implemented)
- [ ] Add `STRIPE_SECRET_KEY_{TENANT}` env var + wire routing (post-Founding-Ten work)

Interim: manual per-tenant Stripe account routing. Not in this cutover.

07 Step 6 · Wire tenant Resend

Code wiring is DONE (2026-07-03). `fromCoach()`, `fromBrand()`, `COACH_BCC` in `src/lib/email-shell.ts` all read from `brand()` / `coach()` — updating the tenant's `brand.fromEmail` via /dashboard/settings/tenant changes the from-address on all 115 send sites automatically.

Remaining manual work per tenant:

- [ ] Add tenant's sending domain to the shared Resend account (Kade's account for now — see scaling note below)
- [ ] Domain verified in Resend (DKIM + SPF DNS records propagated)
- [ ] Test send from `${tenant}@send.${tenant-domain}` to Kade's inbox
- [ ] Verify no delivery to spam (Outlook + Gmail)
- [ ] Update tenant's `brand.fromEmail` and `brand.replyToEmail` via /dashboard/settings/tenant

Scaling note (post-Founding-Ten): All tenants currently share Kade's `RESEND_API_KEY` env var. Kade adds each tenant's DNS records to his Resend account and each tenant sends through him. This works for 10 partners but caps at ~50 verified domains per Resend account. Post-Founding-Ten: introduce per-tenant `RESEND_API_KEY_{TENANT_SLUG}` env vars + a `getResendKey(tenantId)` helper that reads the tenant-scoped key with fallback to the shared key. All 115 send sites already call helper functions, so the choke point is small.

08 Step 7 · Definition of Done

Test each of these on tenant's live domain BEFORE handoff:

- [ ] Homepage loads with THEIR branding (name, logo, colour) — no "Body Recode" visible above the fold
- [ ] Scorecard funnel: entry → quiz → submit → report email fires from THEIR from-address

- [ ] Report email: THEIR logo, THEIR voice, links back to THEIR domain
- [ ] Client can sign up via portal → welcome email from THEIR from-address
- [ ] Foundational intake works + generates CFFS + Foundational Reading
- [ ] Coach can approve/publish → client sees in portal
- [ ] Weekly check-in flow works
- [ ] Payments: test purchase clears in THEIR Stripe
- [ ] Tenant coach logs in as themselves → sees only their clients
- [ ] Kade (as service_role) still has admin access via /dashboard/settings/tenants for support

09 Step 8 · Handover

- [ ] 1-hour training session on the coach dashboard
- [ ] Handover doc: login credentials, how-to-approve-plans, escalation contacts
- [ ] Load in their first 3-5 clients as a real test
- [ ] Tenant runs full loop solo for 24-48h before public launch
- [ ] Kade on standby for launch-day support

---

10 Rollback if something breaks

Any point during onboarding:

1. **Feature flag off:** set `NEXT_PUBLIC_TENANT_DB_ENABLED=false` on Vercel → immediately reverts to in-code BR config for that deployment
2. **Domain redirect:** point their domain at a "coming soon" holding page
3. **Delete tenant row:** `DELETE FROM tenant_config WHERE coach_id = 'THEIR_UUID';` — safe, cache auto-invalidates
4. **Refund test transactions**: from their Stripe (their money, their action)

Zero data loss risk because tenant_config is isolated from client data (which is coach_id-scoped via RLS separately).

---

11 What we're learning from Melisa (pilot zero)

The pilot's real deliverable is a **tightened v2 of this checklist for partners #2-#10.** Track:

- Which manual steps hurt most? (Prime candidates for automation)
- Which fields on `/dashboard/settings/tenant` should have better defaults?
- What did tenant find confusing about the flow?
- What did their clients notice as broken/wrong first?

After Melisa live + stable 30 days → update this checklist → decide GO/NO-GO on partner #2.

---

12 Verification tools

Reusable scripts for verifying the pipeline still works:

- `scripts/test-tenant-pipeline.ts` — 22 unit tests on resolver + loader + cache. Run: `export $(grep -E "^(NEXT_PUBLIC_SUPABASE_URL|SUPABASE_SERVICE_ROLE_KEY)=" .env.local | xargs) && npx tsx scripts/test-tenant-pipeline.ts`
- `scripts/test-melisa-full-pipeline.ts` — end-to-end test: inserts test tenant → verifies loadTenantFromDb → verifies BR row byte-identical → cleans up. Confirms multi-tenant coexistence works.

Run before any Phase 2 deployment as a sanity check.

13 Related

- MELISA_PILOT_ZERO_DEPLOYMENT_RUNBOOK.md — Shape A separate-deploy runbook (still valid for pilot zero)
- PARTNER_JOURNEY.md — 8-stage business process (Attract → Run)
- ../POWERED_PLATFORM_BUILD_PLAN.md — canonical Phase 0-4 plan
- Auto-memory: `project_sot_powered_platform_build_plan`, `project_sot_white_label_licensing_model`